

EECS 31L: INTRODUCTION TO DIGITAL DESIGN LAB LECTURE 6

Salma Elmalaki
salma.elmalaki@uci.edu

The Henry Samueli School of Engineering
Electrical Engineering and Computer Science
University of California, Irvine



LECTURE 6: LOGISTICS

- **All questions should be posted on canvas.**
- Lab 2 is currently being graded - For logistic reasons, any exception requires proofs on record.
- Lab 3 is due on 2/22.
- Please follow the submission guidelines and the lab manual instructions. Failure to follow the instructions in the lab manual will result in losing the lab points. No exceptions!
- Plan your submission ahead of time and do not leave the assignment till the very end.
- **Lab attendance is important -you will need a lot of help from the TAs to do lab 4 and 5**
- **Lab 4 will be assigned after Lab 3 deadline**
- **Midterm is next week.**

MIDTERM FORMAT

- Online (no one in lecture hall to proctor)
- On respondus lockdown browser (mock exam is on canvas)
- Midterm is closed book/closed notes. You don't need extra sheet.
- One hour exam starts at 5pm and ends on 6:00pm - It will have extra credit question.
- The server will open at 4:30 if you want to start early. The server will remain open till 7pm to account for accommodation.
- You can start the exam early but **you can't start later than 5pm** or your exam will not be graded.

PREPARE FOR THE MIDTERM

- Differences between the three types of design description:
 - Dataflow
 - Behavioral
 - Structural
- Assignment with delays in concurrent execution
- Difference between concurrent and sequential execution
- Loops in Verilog
- Blocking vs. non blocking
- Common mistakes in Verilog
- How to write a testbench (initial block, simulate a clock, ...)
- More ...

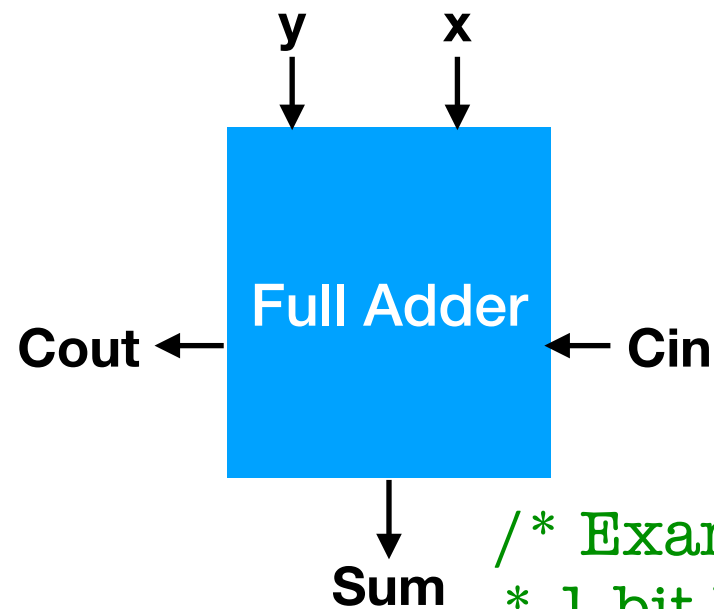
LAST TIME

- Structural description (Chapter 4.1-4.2)
- Binding and port instantiation (Chapter 4.3)
- Examples on blocking vs non-blocking

LECTURE 6: OVERVIEW

- One more example on structural description
- Processor Design Principles (Lab 4, 5)
- Memory Models (Lab 4, 5)

MODULE INSTANTIATION



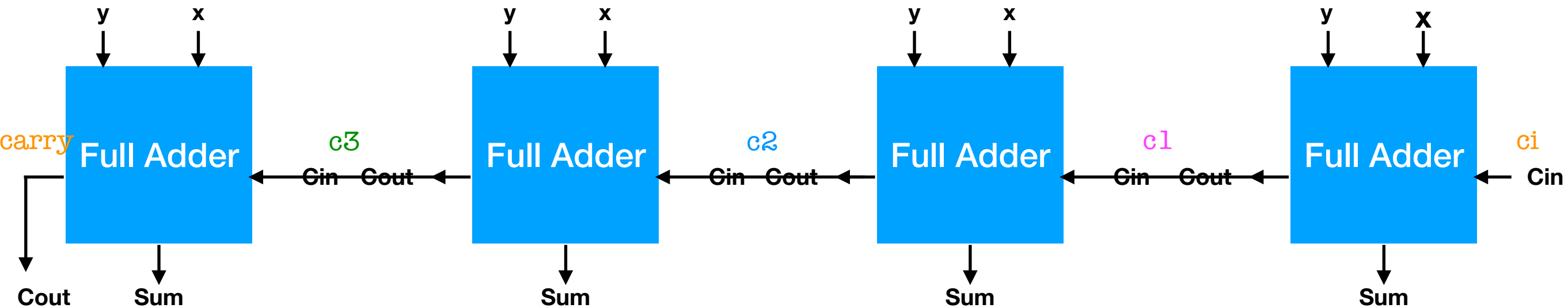
```
/* Example
 * 1-bit FA module code
 */
```

```
module fulladder (input x, input y, input cin,
                  output sum, output cout);
wire m1, m2, m4, m7;
```

```
assign m1 = (~x & ~y & cin);
assign m2 = (~x & y & ~cin);
assign m4 = (x & ~y & ~cin);
assign m7 = (x & y & cin);
assign sum = m1 | m2 | m4 | m7;
assign cout = (x & y) | (cin & (x | y));
endmodule;
```

x	y	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

PORT INSTANTIATION



- Port Association
 - Implicit by order (Position Mapping)
 - Explicit by name (Nominal Mapping)

```

/* 1-bit FA module code */
module fulladder (input x, input y, input cin,
                  output sum, output cout);

wire m1, m2, m4, m7;

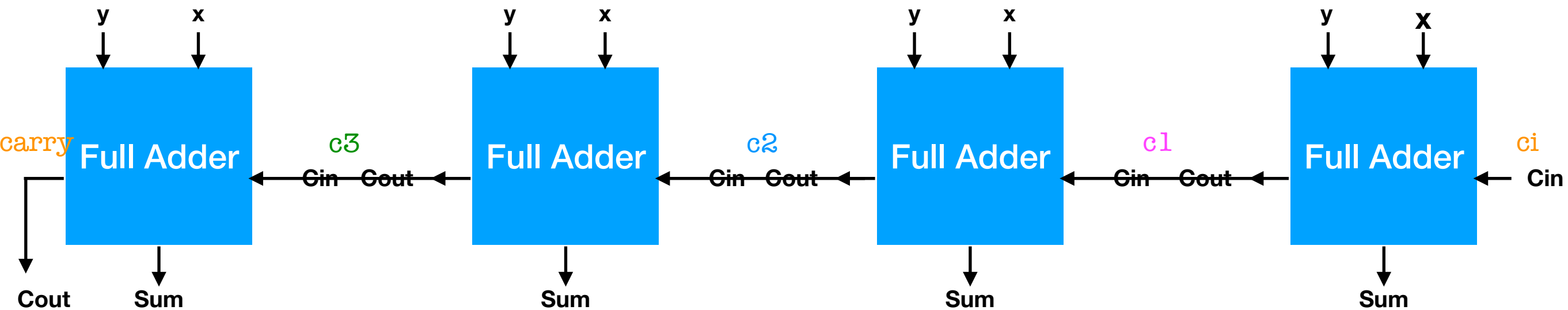
assign m1 = (~x & ~y & cin);
assign m2 = (~x & y & ~cin);
assign m4 = (x & ~y & ~cin);
assign m7 = (x & y & cin);
assign sum = m1 | m2 | m4 | m7;
assign cout = (x & y) | (cin & (x | y));
endmodule;
    
```

```

/* Implicit by Order */
module fa_4bit (result, carry, r1, r2, cin);
// input port declarations
input [3:0] r1 ;
input [3:0] r2 ;
input cin ;
// Output Port Declarations
output [3:0] result ;
output carry ;
wire c1, c2, c3;
fulladder u0 ( r1[0], r2[0], cin, result[0], c1 );
fulladder u1 ( r1[1], r2[1], c1, result[1], c2);
fulladder u2 ( r1[2], r2[2], c2, result[2], c3 );
fulladder u3 ( r1[3], r2[3], c3, result[3], carry );
endmodule;
    
```

The order of the ports match the instantiated module

PORT INSTANTIATION



- Port Association
 - Implicit by order (Position Mapping)
 - Explicit by name (Nominal Mapping)

```
/* 1-bit FA module code */
module fulladder (input x, input y, input cin,
                  output sum, output cout);

wire m1, m2, m4, m7;

assign m1 = (~x & ~y & cin);
assign m2 = (~x & y & ~cin);
assign m4 = (x & ~y & ~cin);
assign m7 = (x & y & cin);
assign sum = m1 | m2 | m4 | m7;
assign cout = (x & y) | (cin & (x | y));
endmodule;
```

```
/* Explicit by name */
fulladder u0 (
    .x (r1[0]),
    .y (r2[0]),
    .cin (ci),
    .sum (result[0]),
    .cout (c1));
fulladder u1 (
    .x (r1[1]),
    .y (r2[1]),
    .cin (c1),
    .sum (result[1]),
    .cout (c2));
fulladder u2 (
    .x (r1[2]),
    .y (r2[2]),
    .cin (c2),
    .sum (result[2]),
    .cout (c3));
fulladder u3 (
    .x (r1[3]),
    .y (r2[3]),
    .cin (c3),
    .sum (result[3]),
    .cout (carry));
```

LECTURE 6: OVERVIEW

- Processor Design Principles (Lab 4, 5)
- Memory Models (Lab 4, 5)

PROCESSOR DESIGN BASICS

- Where to start for designing a processor?
 - Define a language to talk to the processor

- What is a processor's language?

Machine Code

- C/C++/Java?
- Assembly?
- ...?

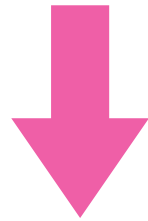
PROCESSOR DESIGN BASICS

```
f = (g + h) - (i + j);
```

C Program

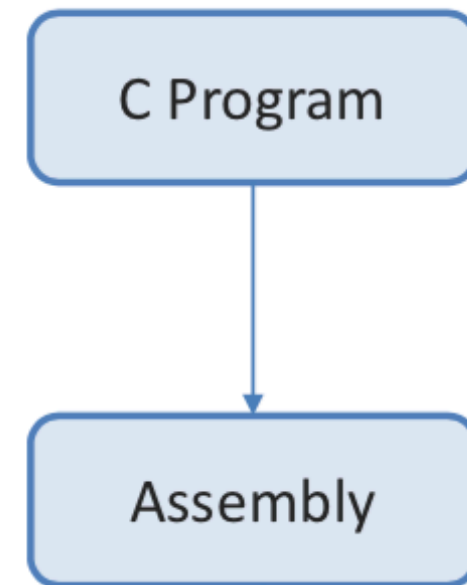
PROCESSOR DESIGN BASICS

`f = (g + h) - (i + j);`



Compiler

```
add t0, g, h    # temp t0 = g + h
add t1, i, j    # temp t1 = i + j
sub f, t0, t1   # f = t0 - t1
```



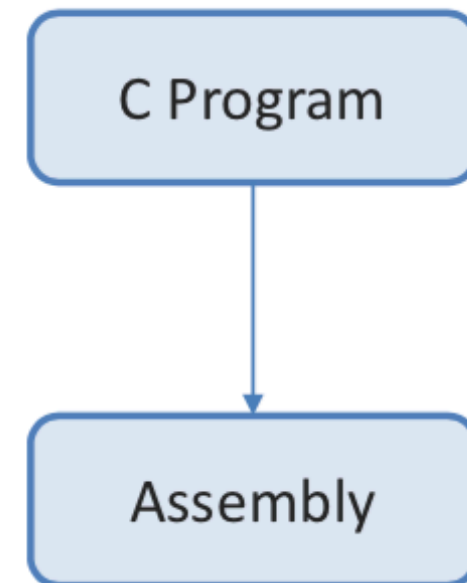
PROCESSOR DESIGN BASICS

`f = (g + h) - (i + j);`



Compiler

```
add t0, R1, R2    # temp t0 = g + h
add t1, R3, R4    # temp t1 = i + j
sub R5, t0, t1    # f = t0 - t1
```



PROCESSOR DESIGN BASICS

`f = (g + h) - (i + j);`



Compiler

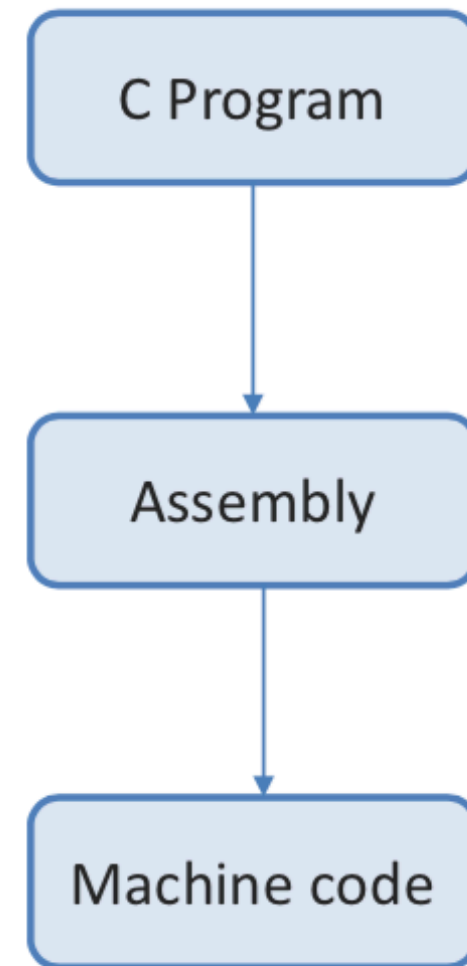
```
add t0, R1, R2    # temp t0 = g + h
add t1, R3, R4    # temp t1 = i + j
sub R5, t0, t1    # f = t0 - t1
```



Assembler

```
1100 0000 0000 0100 0000 1101 1100 0000
1100 0001 0010 0000 0011 1100 0000 0000
0100 0001 0000 0000 0000 1100 0000 0000
```

...



So we need to design a component to get the binary inputs and output the expected results

INSTRUCTION SET ARCHITECTURE (ISA)

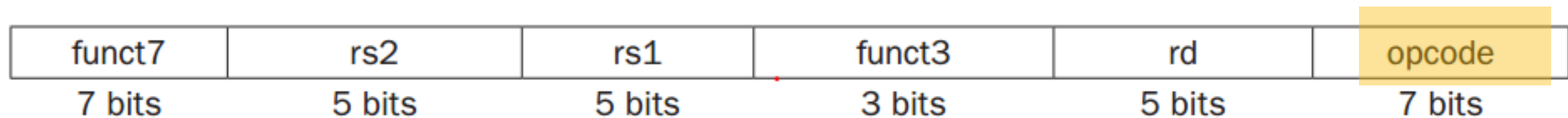
- Includes a specification of:
 - the set of opcodes (**machine language**)
 - the functionalities (**know as instructions**)
implemented for a certain processor.
- You Lab focuses on RISC-V processor.

INSTRUCTION TYPES

- R-type/I-type arithmetic/logic instructions:
 - add, sub, and, or, sll
 - R-type: all the operands are registers (such as add)
 - I-type: one of the operands is a constant (such as addi)
- Memory instructions:
 - lw, sw
- Branches:
 - beq
- Your project does NOT need to support Branch instructions.

HOW THE INSTRUCTION IS INTERPRETED

R-type instruction format



—opcode: Basic operation of the instruction

—rd: destination register index

—funct3 : An additional opcode field

—rs1: first source register index

—rs2 : second source register index

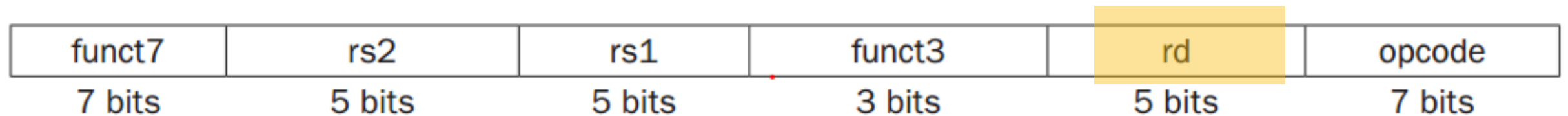
—funct7 : An additional opcode field

Example: $y = x + z$

add r8, r1, r2 #machine code: 0000000 00010 00001 000 01000 0110011

HOW THE INSTRUCTION IS INTERPRETED

R-type instruction format



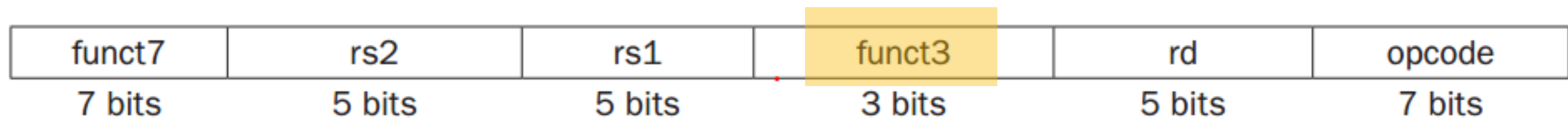
- opcode: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index
- rs2 : second source register index
- funct7 : An additional opcode field

Example: $y = x + z$

add r8, r1, r2 #machine code: 00000000 00010 00001 000 01000 0110011

HOW THE INSTRUCTION IS INTERPRETED

R-type instruction format



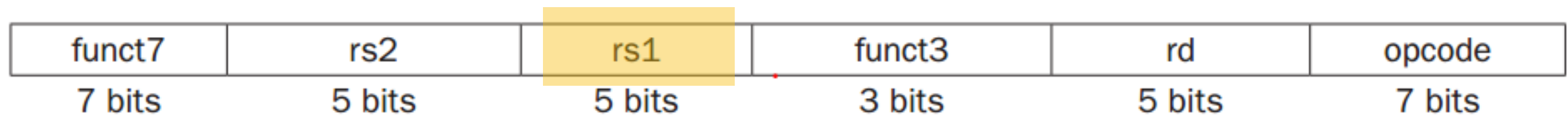
- opcode: Basic operation of the instruction
- rd: destination register index
- funct3: An additional opcode field
- rs1: first source register index
- rs2 : second source register index
- funct7 : An additional opcode field

Example: $y = x + z$

add r8, r1, r2 #machine code: 0000000 00010 00001 000 01000 0110011

HOW THE INSTRUCTION IS INTERPRETED

R-type instruction format



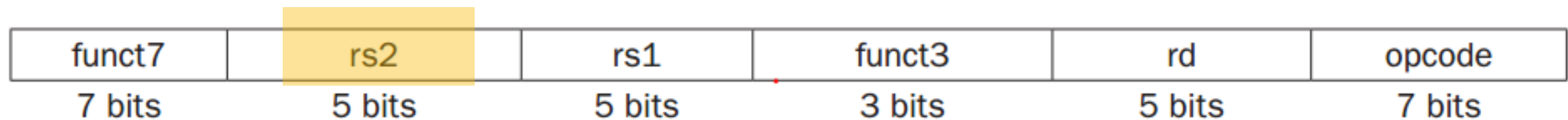
- opcode: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index
- rs2 : second source register index
- funct7 : An additional opcode field

Example: $y = x + z$

add r8, r1, r2 #machine code: 00000000 00010 00001 000 01000 0110011

HOW THE INSTRUCTION IS INTERPRETED

R-type instruction format



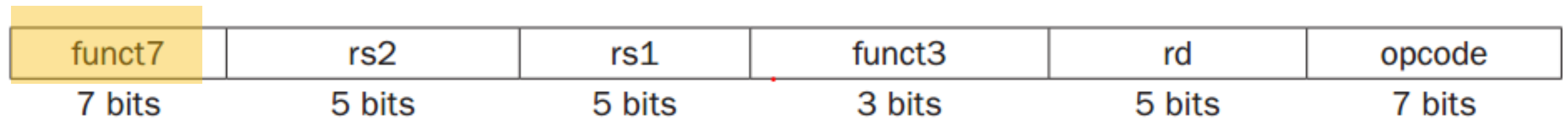
- opcode: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index
- rs2 : second source register index
- funct7 : An additional opcode field

Example: $y = x + z$

add r8, r1, r2 #machine code: 00000000 00010 00001 000 01000 0110011

HOW THE INSTRUCTION IS INTERPRETED

R-type instruction format



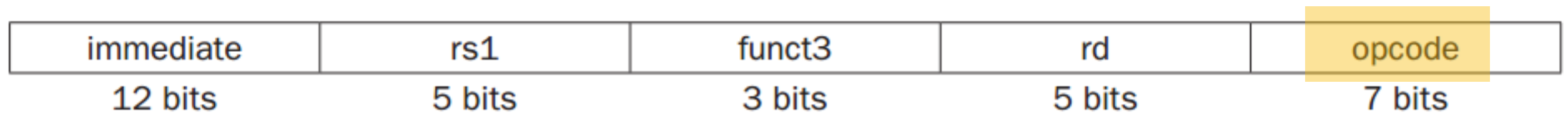
- opcode: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index
- rs2 : second source register index
- funct7 : An additional opcode field

Example: $y = x + z$

add r8, r1, r2 #machine code: 00000000 00010 00001 000 01000 0110011

HOW THE INSTRUCTION IS INTERPRETED

I-type instruction format



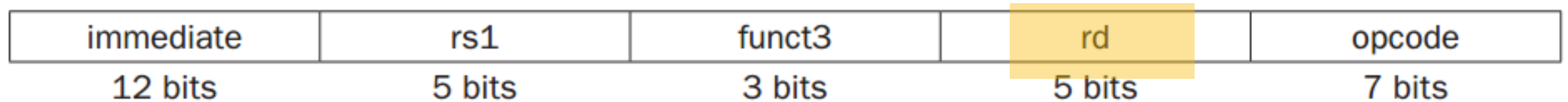
- opcode**: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index
- immediate : 12-bit immediate is interpreted as a two's complement value

Example: $y = x + 4$

addi r4,r1, 4 #machine code: 0000000000100 00001 000 00100 0010011

HOW THE INSTRUCTION IS INTERPRETED

I-type instruction format



- opcode: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index
- immediate : 12-bit immediate is interpreted as a two's complement value

Example: $y = x + 4$

addi r4,r1, 4 #machine code: 0000000000100 00001 000 00100 0010011

HOW THE INSTRUCTION IS INTERPRETED

I-type instruction format



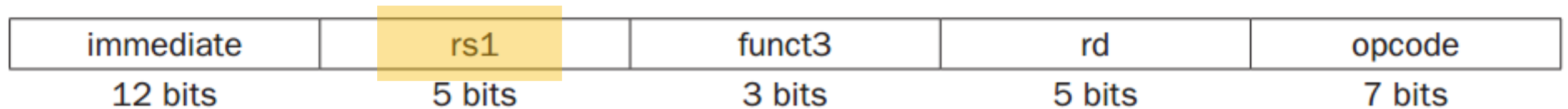
- opcode: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index
- immediate : 12-bit immediate is interpreted as a two's complement value

Example: $y = x + 4$

addi r4,r1, 4 #machine code: 000000000100 00001 000 00100 0010011

HOW THE INSTRUCTION IS INTERPRETED

I-type instruction format



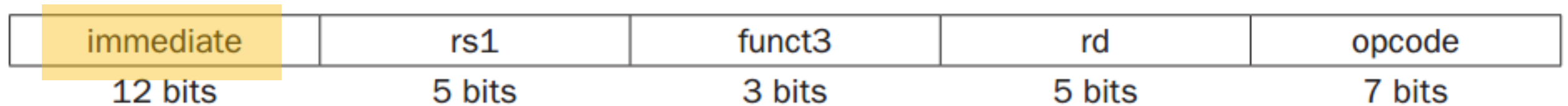
- opcode: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index
- immediate : 12-bit immediate is interpreted as a two's complement value

Example: $y = x + 4$

addi r4, r1, 4 #machine code: 0000000000100 00001 000 00100 0010011

HOW THE INSTRUCTION IS INTERPRETED

I-type instruction format



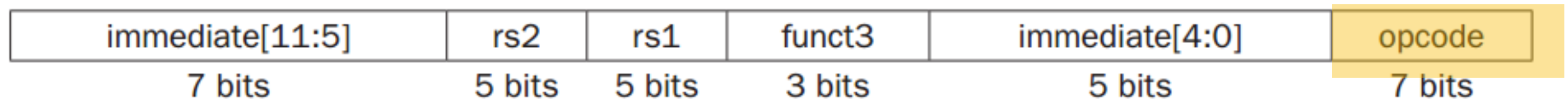
- opcode: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index
- immediate : 12-bit immediate is interpreted as a two's complement value

Example: $y = x + 4$

addi r4,r1, 4 #machine code: **0000000000100 00001 000 00100 0010011**

HOW THE INSTRUCTION IS INTERPRETED

S-type instruction format



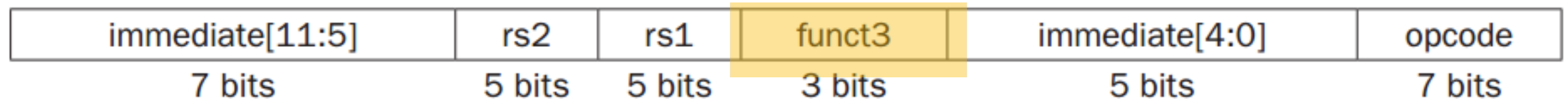
- opcode**: Basic operation of the instruction
- funct3 : An additional opcode field
- rs1: register index to form the relative address
- rs2: register index to be stored
- immediate : 12-bit immediate is split into two fields

Example

sw r10, 48(r0) **#machine code:** 0000001 01010 00000 010 10000 0100011

HOW THE INSTRUCTION IS INTERPRETED

S-type instruction format



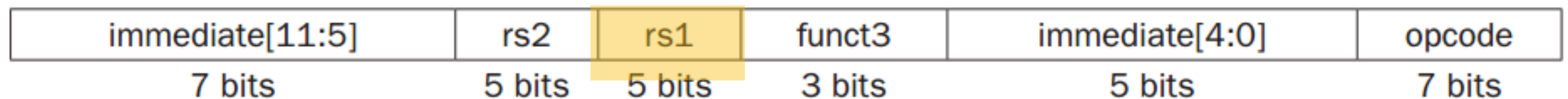
- opcode: Basic operation of the instruction
- funct3 : An additional opcode field
- rs1: register index to form the relative address
- rs2: register index to be stored
- immediate : 12-bit immediate is split into two fields

Example

sw r10, 48(r0) #machine code: 0000001 01010 00000 010 10000 0100011

HOW THE INSTRUCTION IS INTERPRETED

S-type instruction format



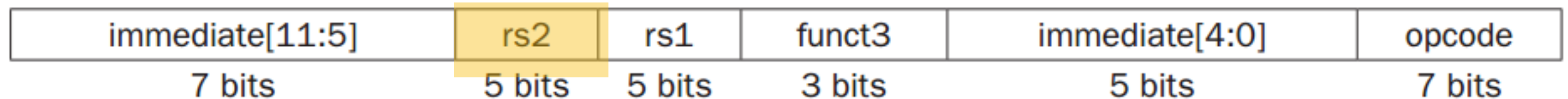
- opcode: Basic operation of the instruction
- funct3 : An additional opcode field
- rs1: register index to form the relative address
- rs2: register index to be stored
- immediate : 12-bit immediate is split into two fields

Example

sw r10, 48(r0) #machine code: 0000001 01010 00000 010 10000 0100011

HOW THE INSTRUCTION IS INTERPRETED

S-type instruction format



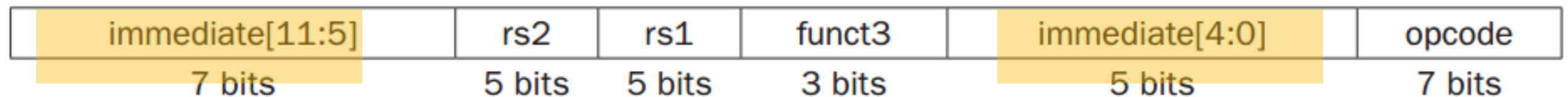
- opcode: Basic operation of the instruction
- funct3 : An additional opcode field
- rs1: register index to form the relative address
- rs2: register index to be stored
- immediate : 12-bit immediate is split into two fields

Example

sw r10, 48(r0) #machine code: 0000001 01010 00000 010 10000 0100011

HOW THE INSTRUCTION IS INTERPRETED

S-type instruction format



- opcode: Basic operation of the instruction
- funct3 : An additional opcode field
- rs1: register index to form the relative address (base)
- rs2: register index to be stored
- immediate : 12-bit immediate is split into two fields

Example

48 = 12'b00000001_10000

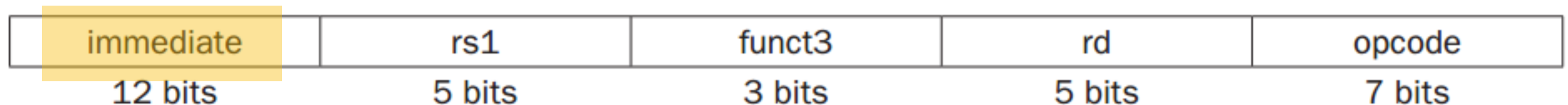
sw r10, 48(r0) #machine code: 00000001 01010 00000 010 10000 0100011

Break 5 minutes
Come at 6:15



HOW THE INSTRUCTION IS INTERPRETED

I-type instruction format - Load



- opcode: Basic operation of the instruction
- rd: destination register index
- funct3 : An additional opcode field
- rs1: first source register index (base)
- immediate : 12-bit immediate

Example: Load

lw r14, 8(r2) #machine code: 000000001000 00010 010 01111 0000011

WHAT YOUR ALU SUPPORTS SO FAR

Table 1 : Instruction Set.

imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	100	rd	0010011	NORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	100	rd	0110011	NOR
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

- You will add the memory operations in lab 4 (sw, lw)

HOW A PROCESSOR WORKS?

1- Reads a instruction from instruction memory
(Fetch)

HOW A PROCESSOR WORKS?

1- Reads a instruction from instruction memory

(Fetch)

2- Interprets the instruction to find out what action should be taken

(Decode)

HOW A PROCESSOR WORKS?

- 1- Reads a instruction from instruction memory
(Fetch)
- 2- Interprets the instruction to find out what action should be taken
(Decode)
- 3- Takes the action as the instruction says
(Execute)

HOW A PROCESSOR WORKS?

- 1- Reads a instruction from instruction memory
(Fetch)
- 2- Interprets the instruction to find out what action should be taken
(Decode)
- 3- Takes the action as the instruction says
(Execute)
- 4- Load or Store from/to the memory
(Memory)

HOW A PROCESSOR WORKS?

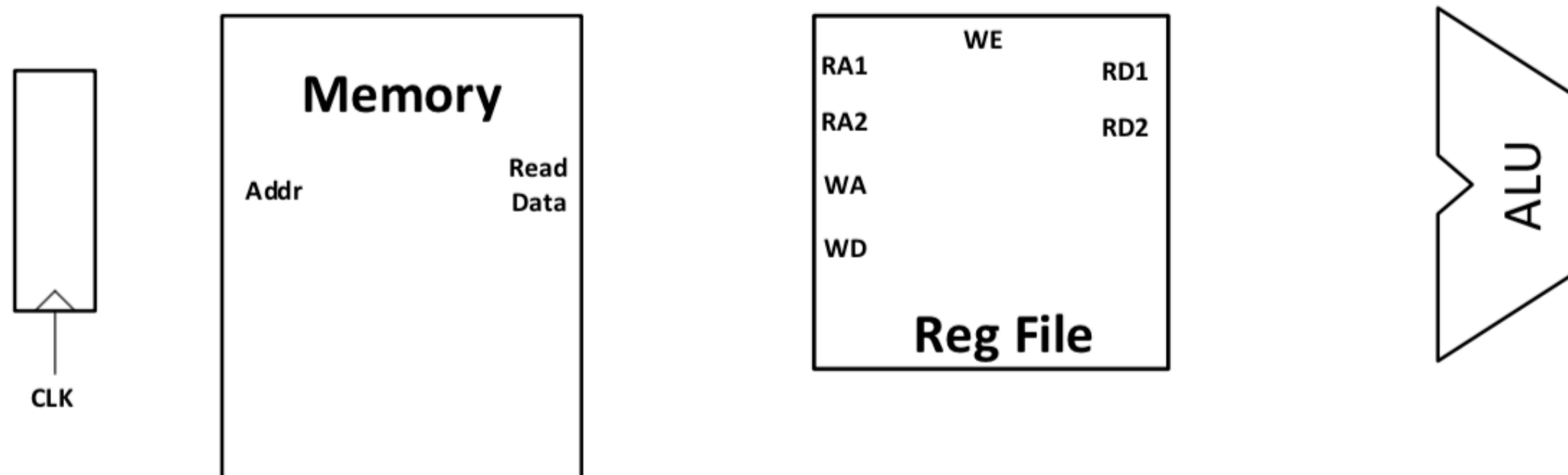
- 1- Reads a instruction from instruction memory
(Fetch)
 - 2- Interprets the instruction to find out what action should be taken
(Decode)
 - 3- Takes the action as the instruction says
(Execute)
 - 4- Load or Store from/to the memory
(Memory)
 - 5- Store the result of execution in the register file
(Write back)
- go to step 1 for executing next instruction

SIMPLE SINGLE-CYCLE PROCESSOR DESIGN

- Using the already designed blocks during lab 2 and 3
 - ALU —> You will be provided with an ALU design
 - Register (FF)
 - Register file
 - Memories (data mem and instruction mem) —> will talk about it today
- The single-cycle architecture executes an entire instruction in one cycle

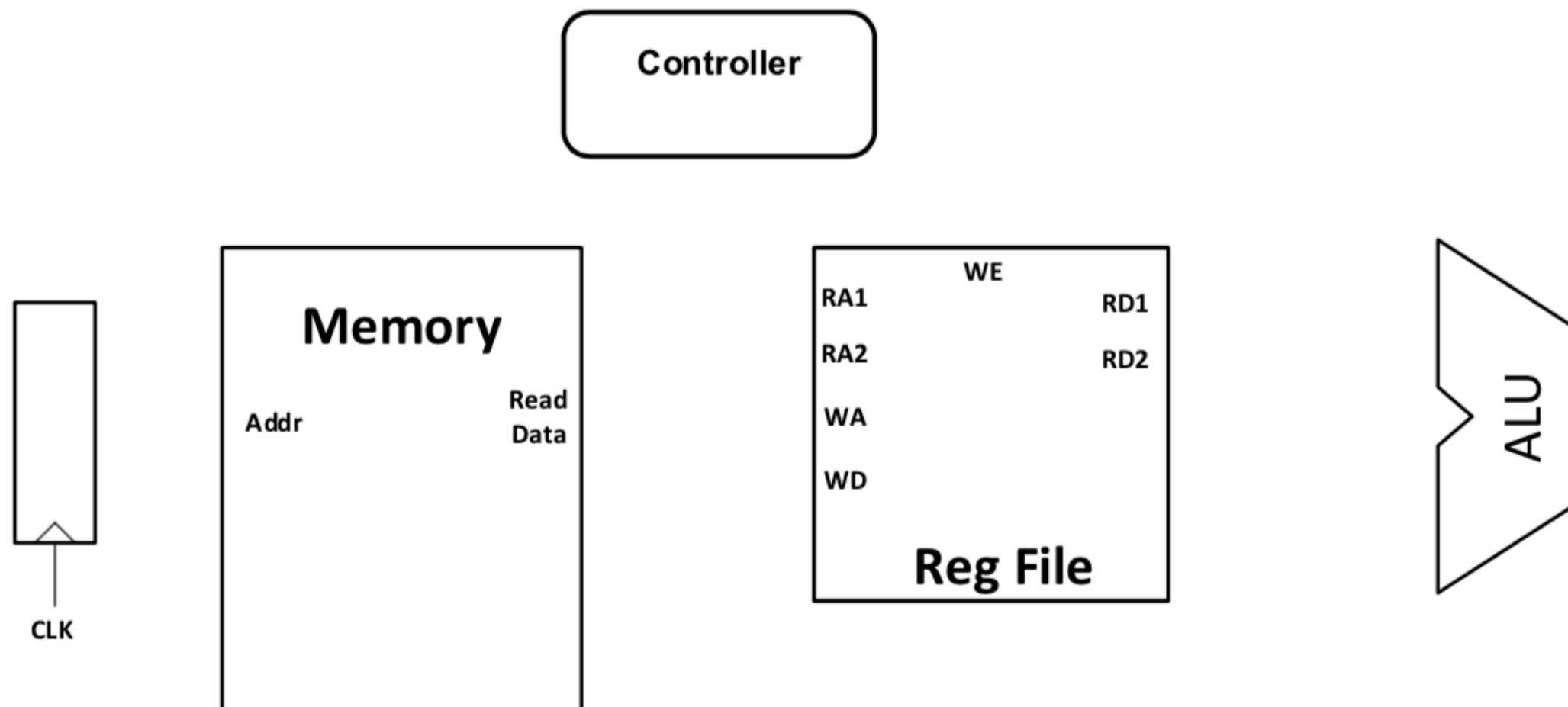
HOW A PROCESSOR WORKS?

- Processor microarchitecture:
 - 1- Datapath (lab 4)



HOW A PROCESSOR WORKS?

- Processor microarchitecture:
 - 1- Datapath (lab 4)
 - 2- Controller (lab 5)



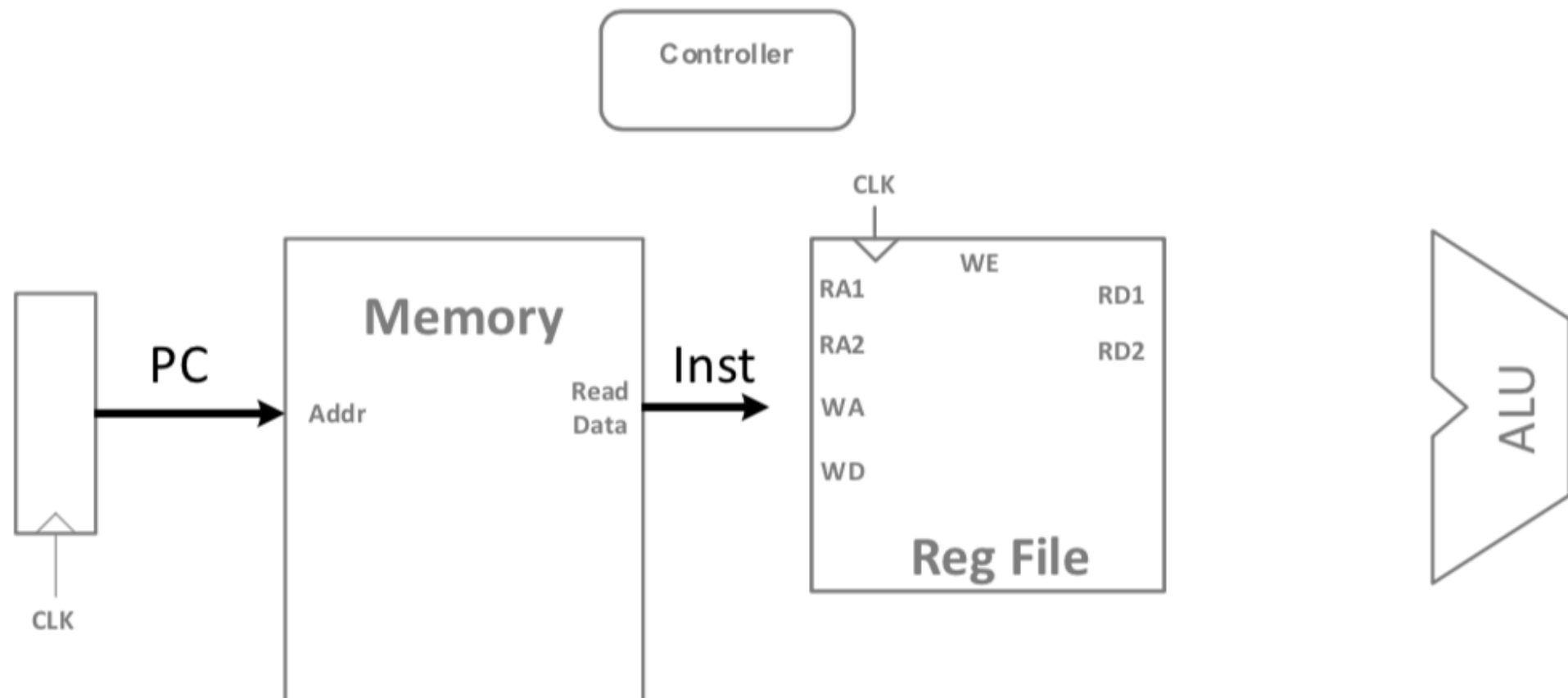
DESIGNING THE DATAPATH (FETCH)

Designing a Single-Cycle Processor Datapath

1- Fetch Stage

The program counter (PC) register contains the address of the instruction to execute.

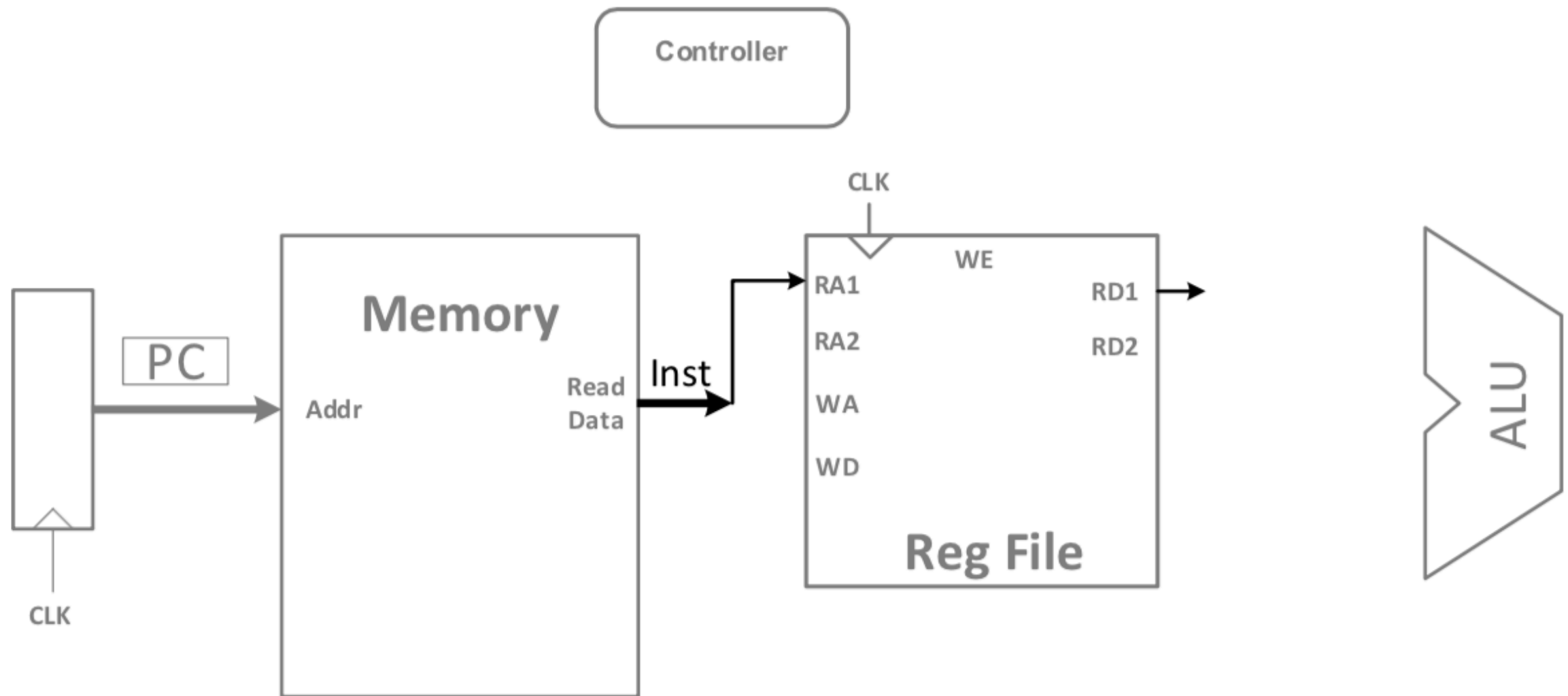
The first step is to read this instruction from instruction memory.



DESIGNING THE DATAPATH (DECODE)

Designing a Single-Cycle Processor Datapath

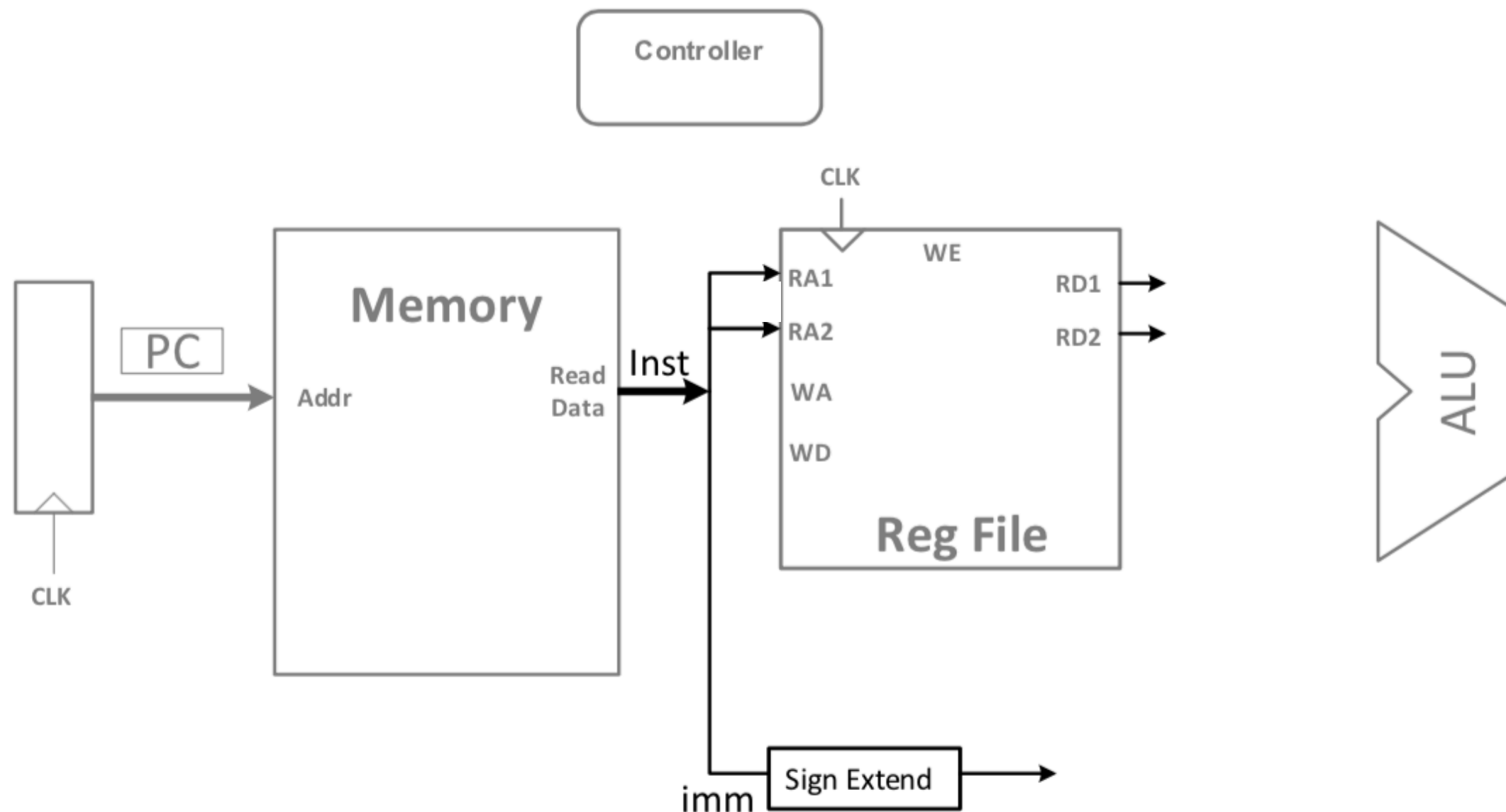
2- Decode Stage



DESIGNING THE DATAPATH (DECODE)

Designing a Single-Cycle Processor Datapath

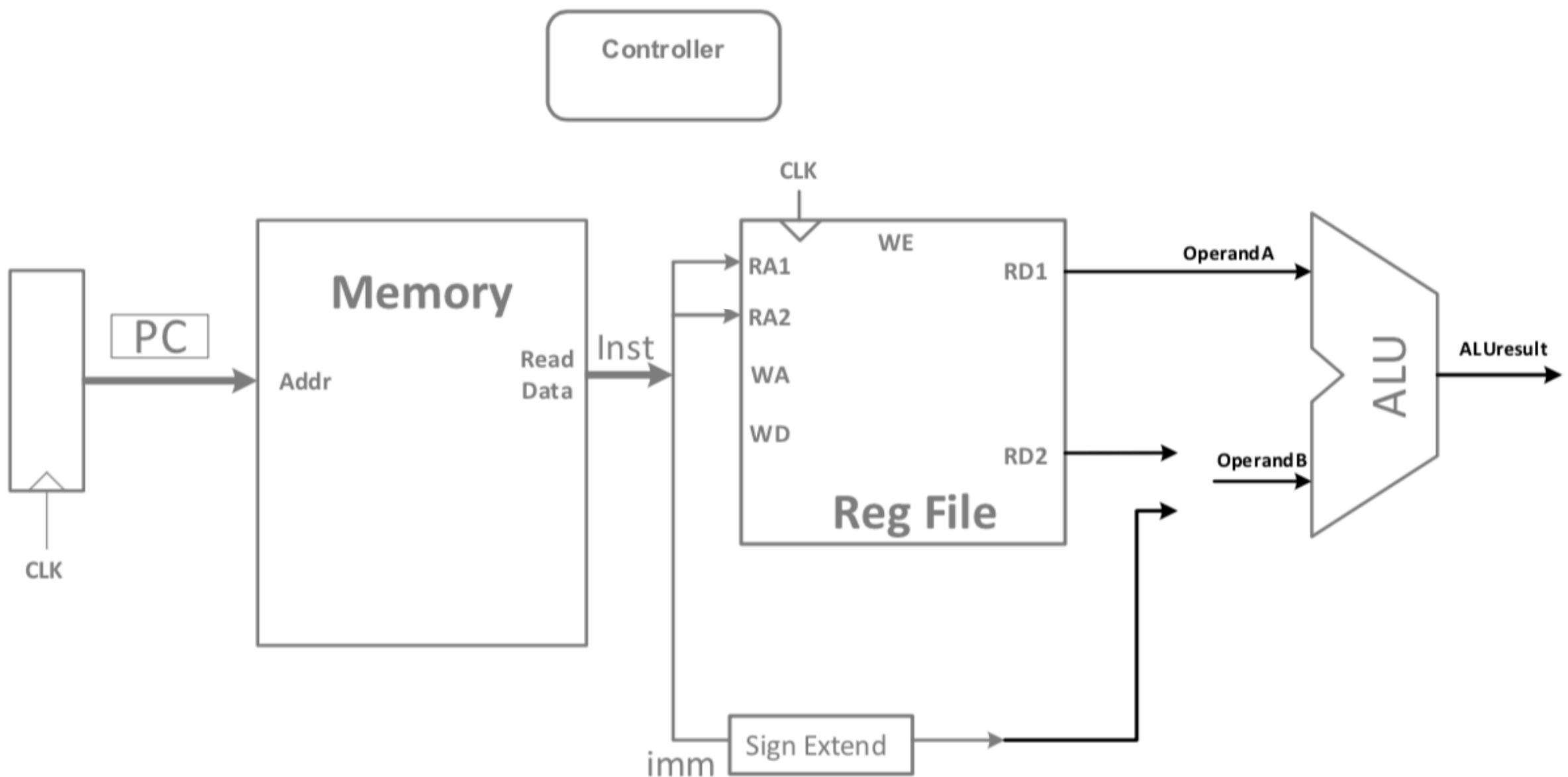
2- Decode Stage



DESIGNING THE DATAPATH (EXECUTE)

Designing a Single-Cycle Processor Datapath

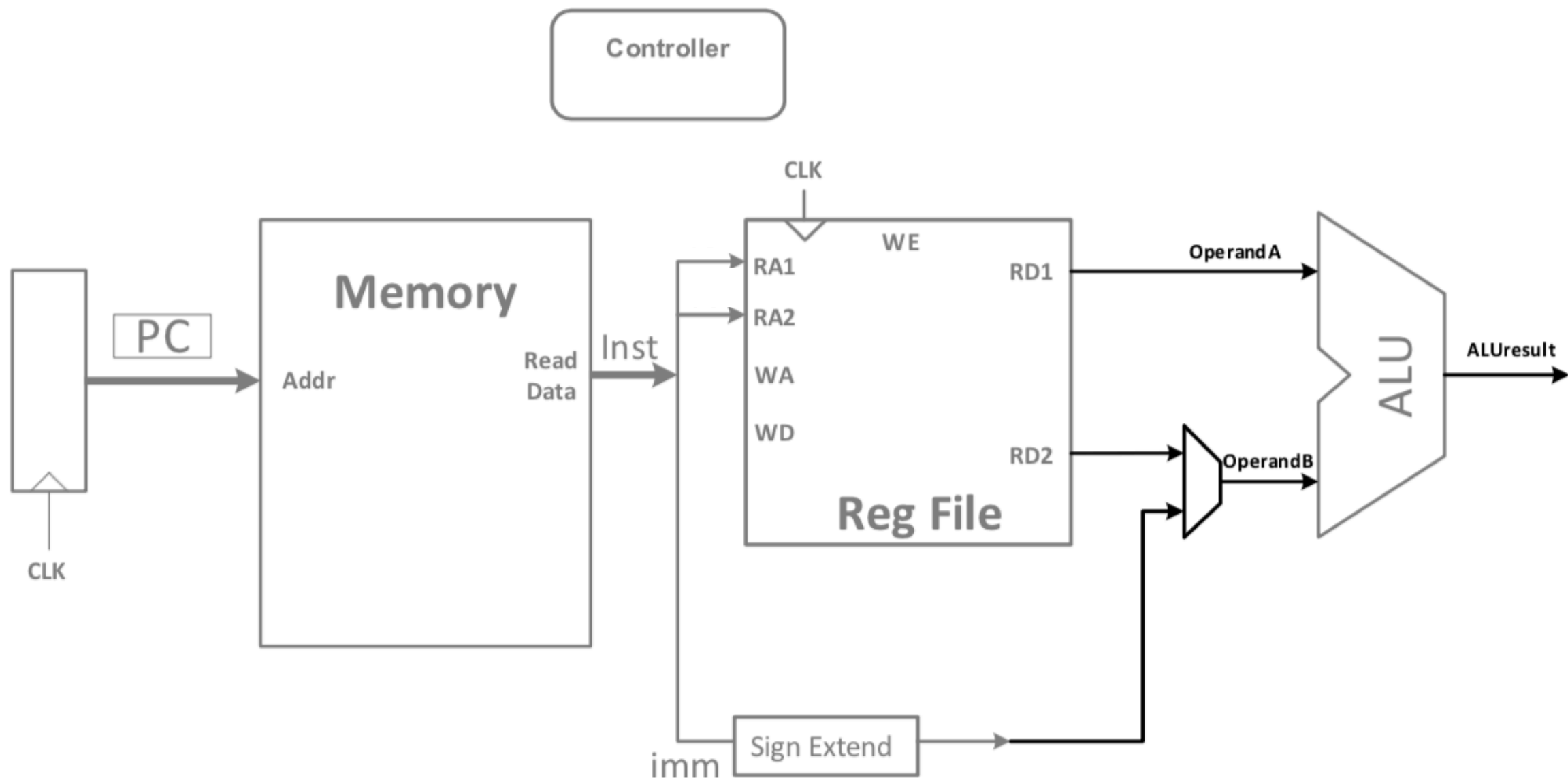
3- Execution Stage



DESIGNING THE DATAPATH (EXECUTE)

Designing a Single-Cycle Processor Datapath

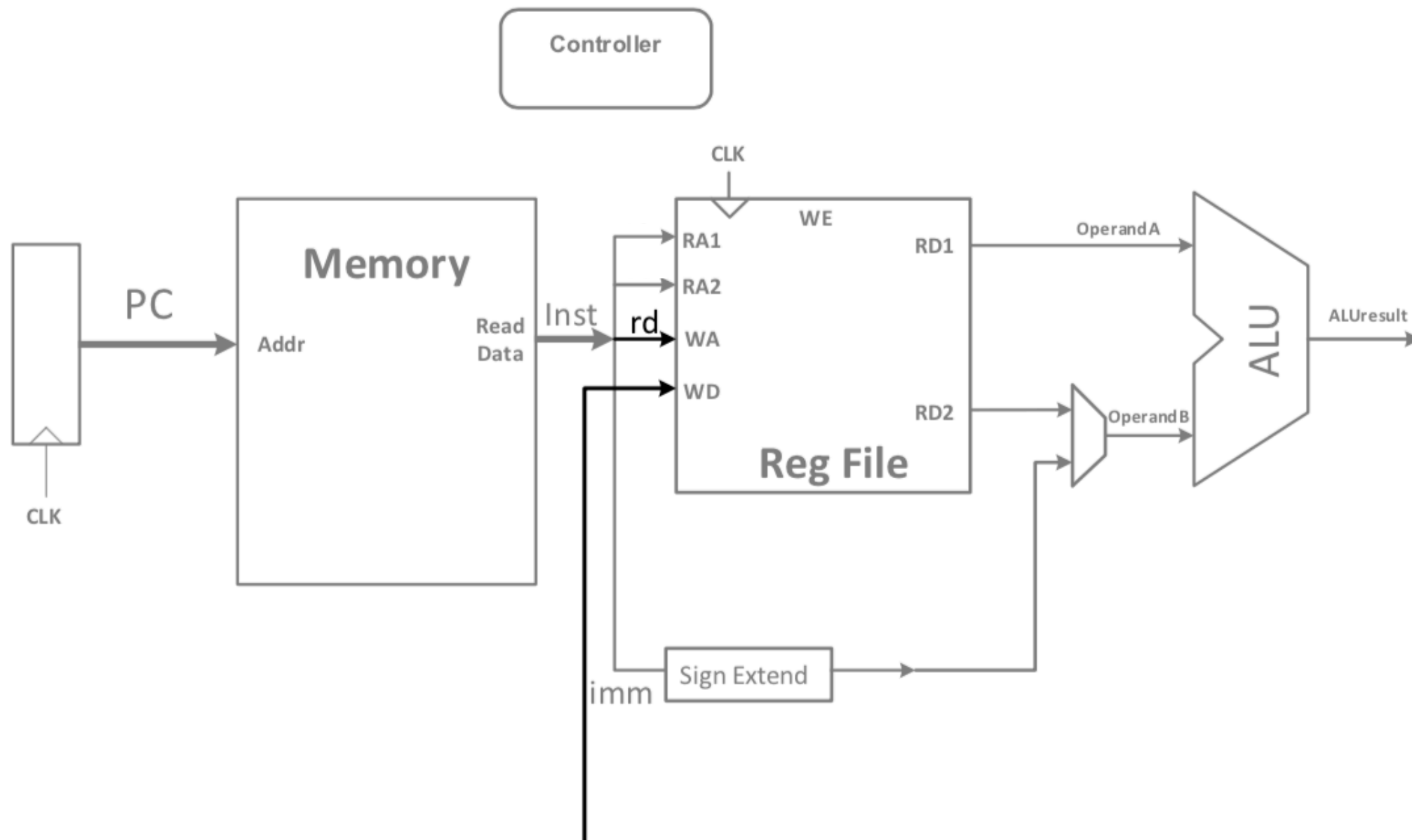
3- Execution Stage



DESIGNING THE DATAPATH (WRITE BACK)

Designing a Single-Cycle Processor Datapath

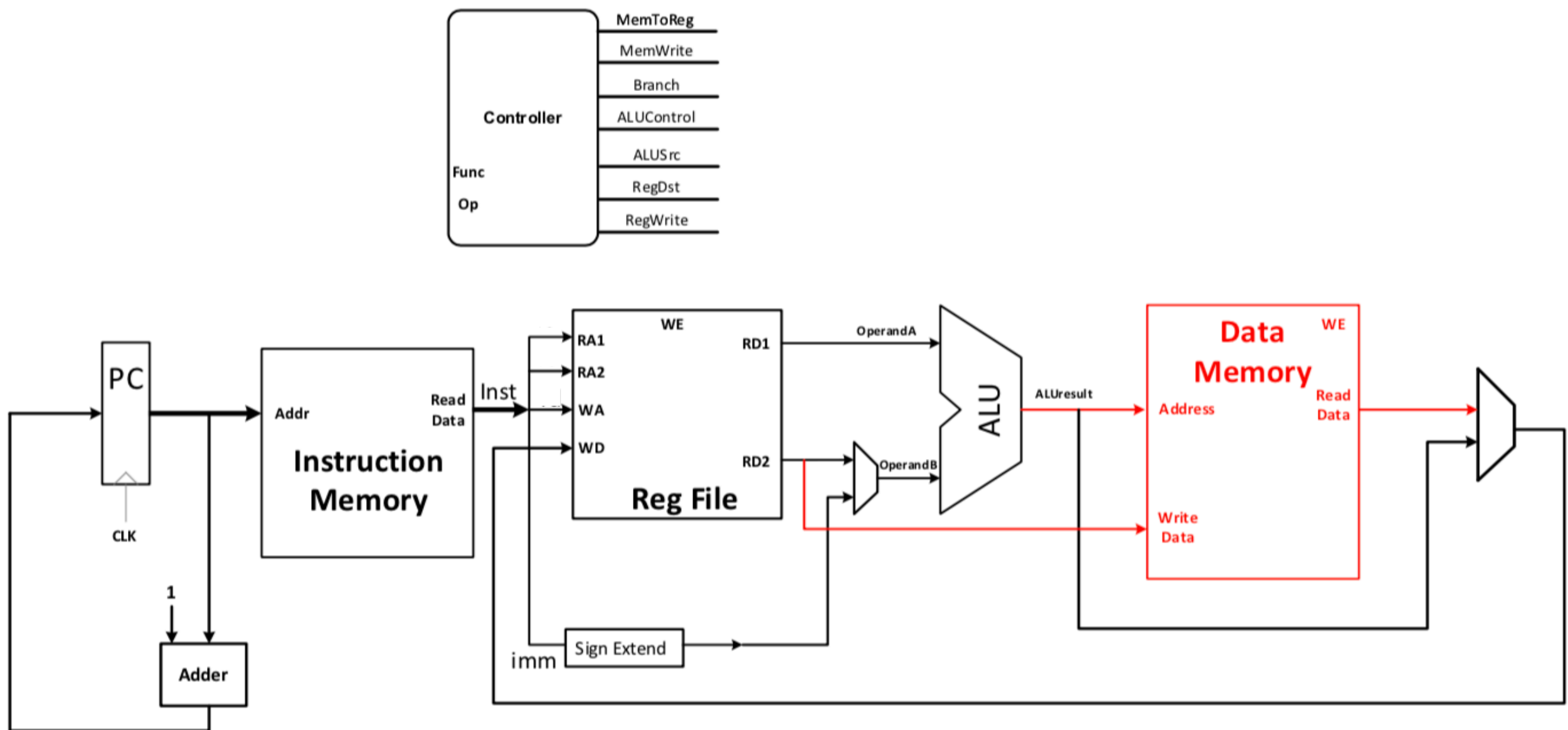
5- Write back Stage



DESIGNING THE DATAPATH (MEMORY)

Designing a Single-Cycle Processor Datapath

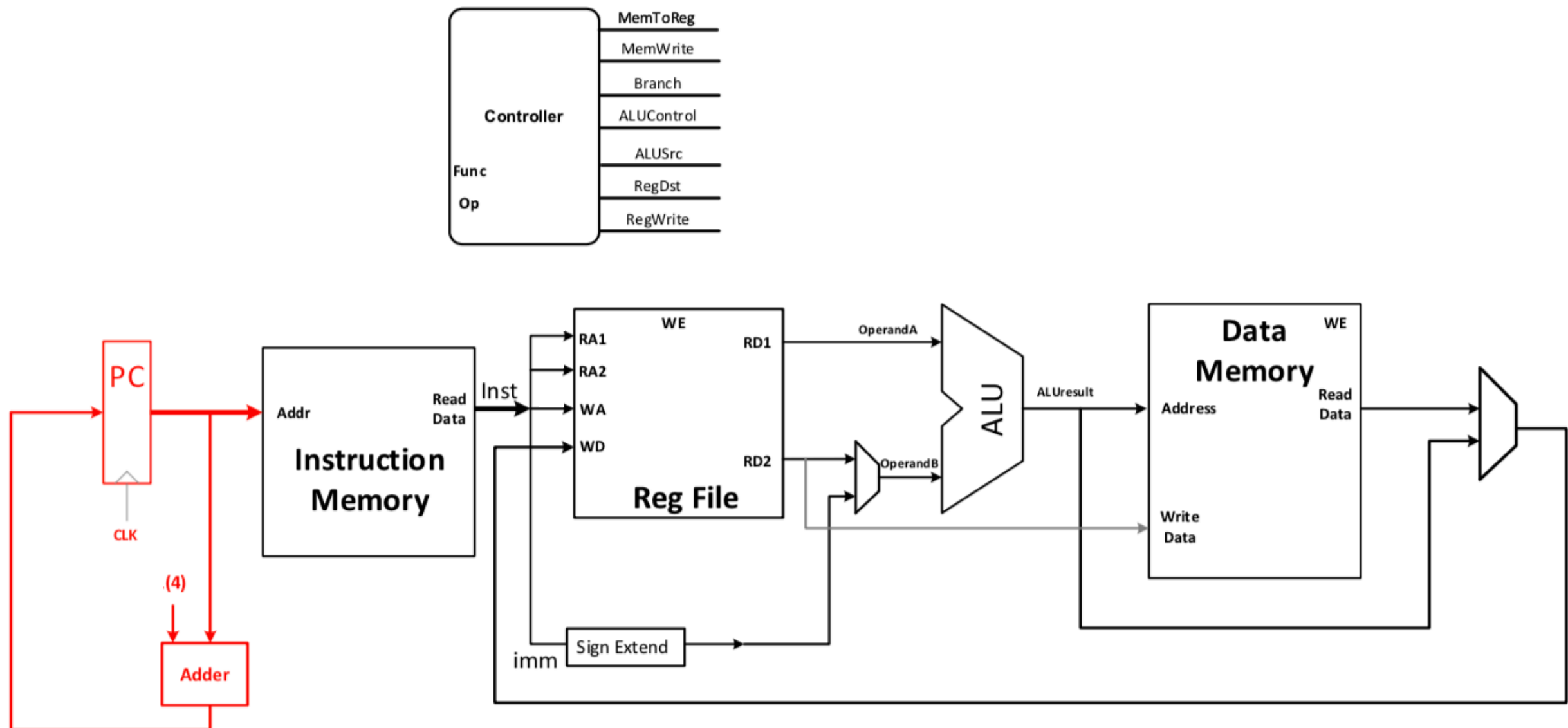
4- Memory



DESIGNING THE DATAPATH (NEXT INSTRUCTION)

Designing a Single-Cycle Processor Datapath

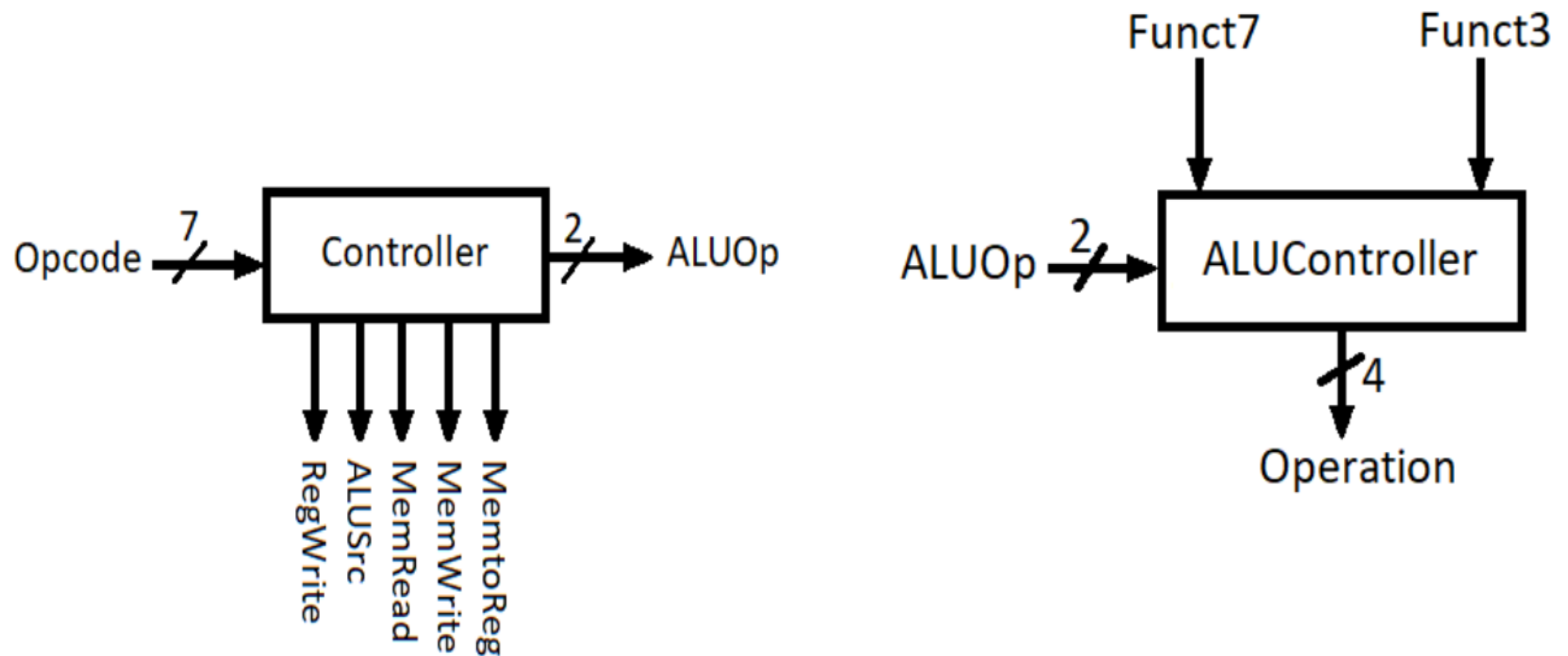
Next Instruction



DESIGNING THE CONTROL PATH (NEXT LAB)

Design controller and ALUController

- generates all controlling signals



DESIGNING THE CONTROL PATH (NEXT LAB)

